

HOODNEST / HOODLINGS

THE SYSTEM ARCHITECTURE & BRAND ARCHITECTURE COMPREHENSIVE BRIEF

Author: Principal Systems Architect & Senior Software Engineer

Version: 2.2.0 (Stable Spec)

Classification: Confidential, Internal Engineering / Marketing Spec

Target Platform: ■ API Integration, Node/NestJS Monolith, Next.js Web App

Date: July 29, 2026

1. Executive Summary & Vision

Hoodnest is an asynchronous, event-driven virtual pet simulation and RPG-lite ecosystem set in a medieval fantasy 'Robin Hood' universe. The system operates natively on **■ (formerly Twitter)** via real-time timeline webhook hooks, and through a high-performance monospaced web portal application.

The core differentiator is **sentience simulation**: digital companions (referred to as **Hoodlings**) are not static chatbots. They possess dynamic state machines, continuous semantic vector memories, distinct personalities that modify RPG encounter outcomes, and are capable of executing autonomous co-op adventures with other companions across the network.

2. Brand Identity & Design Aesthetics

The branding of Hoodnest represents a collision of medieval fantasy folklore and high-fidelity retro command terminals. This identity is structured across three primary layers:

- **Monospaced Minimalist Visuals:** All public dashboards and pages use monospace elements resembling a retro terminal, paired with warm parchment backgrounds (Hex #F7F7F4) and sharp, high-contrast forest neon primary highlights (Hex #4C6B00).
- **The Legendary Companions:** Sherwood Forest is populated by 12 unique animal species, each linked to a tactical guild faction and custom combat/utility role. Factions include *Forest Rangers*, *Recon Corps*, *Shadow Guild*, and *Wisdom & Command Council*.

Name	Species	Faction Group	Class & Duty
Robin Fox	Fox	Forest Rangers	Ranger (Leader, archery, strategist)
Hartley	Deer	Forest Rangers	Hunter (Tracking, long-bow marksman)
Little John	Bear	Forest Rangers	Guardian (Team protector, physical strength)
Harelock	Hare	Recon Corps	Scout (Speedy messenger, map explorer)
Nutley	Squirrel	Shadow Guild	Rogue (Lockpicking, stealth infiltration)
Badgerick	Badger	Logistics & Eng	Quartermaster (Supplies & construction)
Olliver	Owl	Wisdom Council	Sage (Knowledge keeper, strategist)

3. How to Use / Operational Flow

Interacting with Hoodlings is completely frictionless. There are no downloads, installs, or complex login screens. All active management takes place directly on **■ (Twitter)** by writing conversational tags, supported by a **Web Gallery Dashboard** to inspect character states in detail.

A. Core Conversation Interactions on **■**

Users communicate with the bot timeline using normal language. The integrated AI models automatically interpret user intents. The primary interactions are:

- **1. Summoning / Hatching:** To create a pet, a user tweets @HoodNestfun hatch my hoodling or summon a companion. The system checks the user's Twitter ID. If they do not already own a companion, the server assigns a random woodland species with an AI-generated name, a default role, and initial stats. The bot replies with a Welcome Card.
- **2. Feeding:** To feed, the user tags the bot saying @HoodNestfun serve him ramen or give my fox a cookie. The LLM parses the text to identify the food type. It then applies vitals adjustments (lowering Hunger state), writes a contextually accurate response based on the pet's current mood, and replies. Cooldown is set to 60 minutes per feed command.
- **3. Status Checks:** Users can inspect companion vitals dynamically by tweeting @HoodNestfun how is my pet? or check status. The system renders a card displaying Level, EXP, Health, Energy, Hunger, and Friendship values.
- **4. Adventures:** By tweeting @HoodNestfun my companion wants to explore or find a friend, the companion enters a queue. It matches with another user's companion to embark on a shared RPG text adventure. The adventure outputs an event card outlining the narrative, dice rolls, modifiers applied, and results (EXP/item rewards).

B. Web Dashboard Navigation

For deep analysis, users visit the web portal (e.g. hoodlings.xyz). Here they can:

- Authenticate using **Better Auth** via their Google, Discord, or X credentials.
- Access a visual inventory dashboard mapping equipment (such as bows, cloaks, and boots) to overlay on their pixel companion.
- View global ranking leaderboards displaying high-level and evolved companions.
- Check public logs of past collaborative adventures between companions in Sherwood Forest.

4. Dynamic Companion State Machine & Mathematics

Each companion runs on a deterministic tick-based state engine. Below are the formulas and states governing vitals decay, active recovery, and personality influence modifiers during encounters.

A. Vitals State Equations (Ticks defined as T = 1 hour)

1. **Hunger Accumulation (H_t):**

$$H_t = \min(100, H_{t-1} + \Delta H)$$

Where $\Delta H = +5$ per tick. If $H_t \geq 50$, companion becomes Peckish. If $H_t = 100$, status transitions to Famished.

2. **Energy Recovery & Cost (E_t):**

- *Passive Recovery:* $E_t = \min(100, E_{t-1} + 10)$ per idle tick.
- *Action cost:* Conversations cost -5 energy. Adventures cost -20 energy.

3. **Health Decay (HP_t):**

If companion is Famished (Hunger = 100):

$HP_t = \max(0, HP_{t-1} - 10)$ per tick. At $HP = 0$, companion enters the Dead state and must be revived with a Revive Potion.

B. Personality Modifier Values

A companion's personality acts as a coefficient vector applied to RPG dice rolls during adventure events:

$$Roll = Base\ D20 + Stat\ Modifier + \delta_{personality}$$

Personality	Primary Mod	Positive Modifier ($\delta+$)	Negative Modifier ($\delta-$)
Brave	Strength	+4 on combat encounters	-2 on stealth traps
Wise	Intelligence	+4 on riddle traps	-2 on physical combat
Curious	Luck	+3 on discovery/loot	+2 encounter danger rate
Lazy	Energy	-50% passive energy decay	-3 on speed check quests

5. Event-Driven Webhook & Queue Infrastructure

To scale timeline interactions without hitting API rate limits or losing messages under heavy load, the backend runs a decoupled event gateway using NestJS, Redis, and BullMQ.

A. Cryptographic Signature Verification

To verify incoming payloads from ■ and prevent replay attacks, the server executes HMAC SHA256 validation:

```
import * as crypto from 'crypto';
export function verifyXSignature(payload: string, signature: string, clientSecret: string): boolean {
  const hmac = crypto.createHmac('sha256', clientSecret).update(payload).digest('base64');
  return crypto.timingSafeEqual(Buffer.from(hmac), Buffer.from(signature));
}
```

B. Concurrency Control & State Locking

To prevent race conditions (e.g. double-feeding or simultaneous adventure actions), we execute distributed locks using Redis-based Redlock keys:

```
const lockKey = `lock:companion:${companionId}`;
const lock = await this.redis.set(lockKey, 'locked', 'PX', 5000, 'NX');
if (!lock) {
  throw new ConflictException('Companion is currently processing another action.');
```

6. AI Prompting, Memory & Vector Search (RAG)

Companions store and retrieve memories dynamically. Customer queries are embedded and matched against historical conversational vectors using pgvector cosine distance metrics:

```
SELECT id, memory_key, memory_value, 1 - (embedding <=> :queryEmbedding) AS similarity
FROM "CompanionMemory"
WHERE "companionId" = :companionId AND 1 - (embedding <=> :queryEmbedding) > 0.78
ORDER BY similarity DESC
LIMIT 3;
```

7. Card Rendering Engine

Instead of simple text replies, Hoodlings are generated as layered transparent PNG sprites and merged onto a custom card template (1200x675 px) before being uploaded to Twitter.

Asset Stack Layers:

1. **Base Body Layer:** Unique woodland creature outline frames (Fox, Stag, Badger, etc.)
2. **Outfit Overlay Layer:** Outlaw jubah and hoods matching evolution levels (Hatchling, Scout, Guardian).
3. **Weapon Overlay Layer:** Transparent attachments mapping to equipped weapons (Bows, Staffs).

Backend canvas composition snippet:

```
const canvas = createCanvas(1200, 675);
const ctx = canvas.getContext('2d');
// Render background template
ctx.drawImage(await loadImage('./assets/card_base.png'), 0, 0);
// Compose layered companion frames
ctx.drawImage(await loadImage(`./assets/pets/${species}/base.png`), 450, 100, 300, 300);
ctx.drawImage(await loadImage(`./assets/pets/${species}/outfit_stage_${evolution}.png`), 450, 100, 300, 300);
return canvas.toBuffer('image/webp');
```

8. Database Schema Reference (Prisma)

The underlying models mapped in PostgreSQL using Prisma handle User configurations and companion status logs:

```
model Companion {
  id String      @id @default(uuid())
  userId String
  user User      @relation(fields: [userId], references: [id])
  name String
  species String
  personality String
  evolutionLvl Int @default(1)
  level Int       @default(1)
  energy Int       @default(100)
  health Int       @default(100)
  hunger Int       @default(0)
  strength Int     @default(10)
  intelligence Int @default(10)
  luck Int         @default(10)
}
```